

Умова задачі

Зробити програму, яка моделює **редактор** для створення та редагування табло автовокзалу. На Табло повинен бути заголовок, та список рейсів з відповідною інформацією(номер зупинки, час, номер автобуса і тд).

Функціональність

Загальне призначення: Програма імітує редактор для створення або редагування вигляду інформаційного електронного табло, відображаючи структурований розклад відправлення автобусів, заголовок та час(задається вручну).

1. Редагування інформації

Основне завдання програми — додавання, редагування та видалення наступних об'єктів з екрану:

- довільного тексту(заголовку, назв колонок, ...)
- ліній для розмітки(горизонтальних або вертикальних)
- слотів розкладу(час відправлення, номери автобуса та зупинки, статус рейсу)

2. Зчитування та зберігання Табло

Програма може зчитувати вже існуюче табло з .txt файлу. Також є можливість створити нове Табло(новий файл) та зберегти зміни у файл. Також існує файл config.txt в якому можна задати параметри програми - максимальний розмір текстових полів, розміри та фоновий символ екрану. Ще там зберігається відступи між кожним полем слоту(не зовсім так, але я не знаю як сформулювати)

3. Виведення Табло в термінал

Для перевірки змін програма виводить (та додатково може виводити за командою) табло в термінал. Також при виводі табло в термінал програма перевіряє об'єкти на коректність (коректність полів структури Спрайт) та на перетини між собою та з границями екрану.

Приклад, як може виглядати табло:

Р О З К Л А Д				12:45
ПЛАТФОРМА	ЧАС ВІДПРАВКИ ЗА РОЗКЛАДОМ	ПЕРЕДБАЧУ- ВАНИЙ ЧАС ВІДПРАВКИ	НОМЕР РЕЙСУ	СТАТУС
4	12 : 50	12 : 52	744	ОЧІКУЄТЬСЯ
5	13 : 00	----	87	СКАСОВАНО
2	13 : 04	13 : 04	395	ОЧІКУЄТЬСЯ
11	13 : 20	13 : 30	52	ОЧІКУЄТЬСЯ
7	13 : 44	----	87	СКАСОВАНО
4	13 : 50	13 : 50	744	ОЧІКУЄТЬСЯ
6	14 : 10	14 : 10	285	ОЧІКУЄТЬСЯ
9	14 : 15	----	96	СКАСОВАНО
11	14 : 20	14 : 20	52	ОЧІКУЄТЬСЯ

Вибір Структур

Для зручного створення та змінення вигляду інформації на екрані, зручно будь який об'єкт, будь то заголовок, лінія або пункт розкладу, на екрані назвати спрайтом(Sprite). В кожного спрайту буде своя назва і, що головніше, координати. Далі спрайти будуть відрізнятися - 3 різні види (заголовок, лінія

або слот розкладу). Ці різні види буде зручно об'єднати в union. Далі буде зазначені всі необхідні структури з коментарями:

```
struct Text{ //структура яка відповідає тексту
    char text[MAX_SIZE];
};

typedef enum{      //напрями лінії(вона може бути як вертикальна, так і
горизонтальна)
    RIGHT,
    DOWN
} DirectionEnum;

struct Line {      //структура яка відповідає лінії
    char character;
    int length;
    DirectionEnum direction;
};

typedef enum {      //статус рейсу
    WAITING,        //автобус ще не відправився
    IN_PROGRESS,    //автобус вже виїхав
    CANCELLED//рейс скасовано
} SlotStatus;

typedef struct{      //структура для збереження моменту часу
    int hour;
    int minute;
} TimeType;

typedef struct Slot {      //структура яка відповідає одному слоту в розкладі(рейсу)
    int trip_number;
    int station;

    SlotStatus status;
    TimeType scheduled_departure;    //запланований час відправлення
    TimeType estimated_departure;    //передбачуваний час відправлення
} ScheduleSlot;

typedef enum {      //тип спрайту
    TEXT,
    LINE,
    SLOT
} TypeOfSprite;

typedef union {      //об'єднання всіх типів спрайту
    struct Text text;
    struct Line line;
    struct Slot slot;
} DetailsType;
```

```
typedef struct { //структура яка відповідає спрайту
    char name[MAX_SIZE];
    int x;
    int y;

    TypeOfSprite type;
    DetailsType details; //поле де зберігається інформація відповідного типу спрайту
} SpriteType;
```

Функції:

```
//////////info input/output functions//////////
/*
 * функція для введення конкретного спрайту.
 * Отримує на вхід вказівник на спрайт, який треба заповнити
 */
void input_sprite(SpriteType *sprite);
/*
 * допоміжна функція, яка вводить деталі в залежності від конкретного типу спрайту
 * Отримує на вхід вказівник на деталі спрайту, які треба заповнити, та тип спрайту.
 */
void input_details(DetailsType *details, TypeOfSprite type);

/*
 * функція для виводу всієї інформації, що зберігається в спрайті, в термінал
 * Отримує на вхід вказівник на спрайт, інфу якого треба вивести
 */
void output_sprite(SpriteType *sprite);
/*
 * допоміжна функція, яка виводить деталі в термінал в залежності від конкретного типу
спрайту
 * Отримує на вхід вказівник на деталі спрайту, які треба вивести, та тип спрайту.
 */
void output_details(DetailsType *details, TypeOfSprite type);

//////////functions for 1 Sprite//////////
/*
 * функція для виводу необхідної інформації, що зберігається в спрайті, на екран
 * Отримує на вхід вказівник на спрайт, інфу якого треба вивести
 */
void print_sprite(SpriteType *sprite);
/*
 * допоміжна функція, яка виводить деталі на екран в залежності від конкретного типу
спрайту
 * Отримує на вхід вказівник на деталі спрайту, які треба вивести, та тип спрайту
 */
void print_details(DetailsType *details, TypeOfSprite type);
/*
 * допоміжна функція, яка виводить текстовий спрайт на екран
 * Отримує на вхід вказівник на спрайт, інфу якого треба вивести
 */
void print_text(SpriteType *sprite);
/*
 * допоміжна функція, яка виводить лінію на екран
 * Отримує на вхід вказівник на спрайт, інфу якого треба вивести
 */
```

```

*/
void print_line(SpriteType *sprite);
/*
* допоміжна функція, яка виводить слот розкладу(рейс) на екран
* Отримує на вхід вказівник на спрайт, інфу якого треба вивести
*/
void print_slot(SpriteType *sprite);

/*
* функція, яка імітує виїзд автобуса. Тобто встановлює реальний час відправлення
відповідному рейсу
* Отримує на вхід спрайт та час відправлення
*/
void set_actual_departure_time(SpriteType *sprite, TimeType depTime);
//void bus_arrived(SpriteType *sprite, TimeType time);

/*
* функція, яка підраховує розмір спрайту. Тобто сліки місця він займає на екрані. В
нашій моделі(імітації) це буде кількість символів, бо замість графічного інтерфейсу
буде використовуватись консоль
* Отримує на вхід відповідний спрайт
* Повертає ціле число - довжину спрайту
*/
int measure_sprite_length(SpriteType *sprite);

/*
* функція, яка перевіряє коректність спрайту. Тобто чи задані всі поля(крім реального
часу відправлення), та чи задовольняють вони допустимим значенням
* Отримує на вхід відповідний спрайт
* Повертає true, якщо спрайт коректний, та false в іншому випадку
*/
bool check_sprite(SpriteType *sprite);

/*
* допоміжна функція, яка перевіряє коректність деталей, відповідно від типу спрайта
* Отримує на вхід вказівник на деталі спрайту, які треба вивести, та тип спрайту
* Повертає true, якщо деталі коректні, та false в іншому випадку
*/
bool check_details(DetailsType *details, TypeOfSprite type);

//////////functions for more than one Sprite//////////
/*
* функція, яка серед масиву спрайтів знаходить спрайти, які відповідають макету. Тобто
поля який дорівнюють не заблокованим полям макетного спрайту.
* Отримує на вхід масив спрайтів(вказівник на перший елемент), та макетний спрайт
* Повертає масив спрайтів(вказівник на перший елемент масиву)
*/
SpriteType *find_sprites_by_model(SpriteType *sprites, SpriteType model);

/*
* Функція яка блокує всі поля спрайту, в залежності від його типу. Саме поле, яке
показує тип спрайту, не блокується
* Отримує вказівник на відповідний спрайт
*/
void block_sprite_fields(SpriteType *sprite);
/*

```

```
* Допоміжна функція яка порівнює спрайт з макетним спрайтом
* Отримує вказівник на відповідний спрайт та макетний спрайт
* Повертає true, якщо спрайт відповідає макету, і false в іншому випадку
*/
bool cmp_sprite_with_model(SpriteType *sprite, SpriteType model);

/*
* Функція, яка серед масиву спрайтів знаходить спрайти, які повністю(початок і кінець)
знаходяться в заданому діапазоні(прямокутнику) .
* Отримує на вхід масив спрайтів(вказівник на перший елемент) та дві пари координат -
лівий верхній та правий нижній кути діапазону
* Повертає масив спрайтів(вказівник на перший елемент масиву)
*/
SpriteType *find_sprites_by_coords_diapason(SpriteType *sprites, int x1, int y1, int
x2, int y2);

/*
* Функція, яка перевіряє, чи не накладаються спрайти один на одний
* Отримує на вхід вказівники на 2 спрайти
* Повертає true, якщо спрайти перетинаються, і false в іншому випадку
*/
bool check_collision(SpriteType *sprite1, SpriteType *sprite2);
```